Al al-Bayt University - Prince Al Hussein bin Abdullah II Faculty of Information Technology

Course Information and Database Security (906313)
First Semester 2026/2027 - Final Semester Project

SecARS

Secure Academic Records System

Architecture Diagram and Data Flow Diagram

System architecture, data flows and trust boundaries

Group members
Abdullah Ibrahim Hassan
Abdulrahman Al-Otaibi

Milestone M1 - Analysis & Design | Due Friday 06/11/2026
Version 1.0 | November 2026

1. Architecture diagram

SecARS runs entirely on one local lab machine, but it is designed as separate zones with explicit trust boundaries (TB1-TB4, red arrows). Nothing that crosses a boundary is trusted until it has been authenticated, authorised and validated on the receiving side.

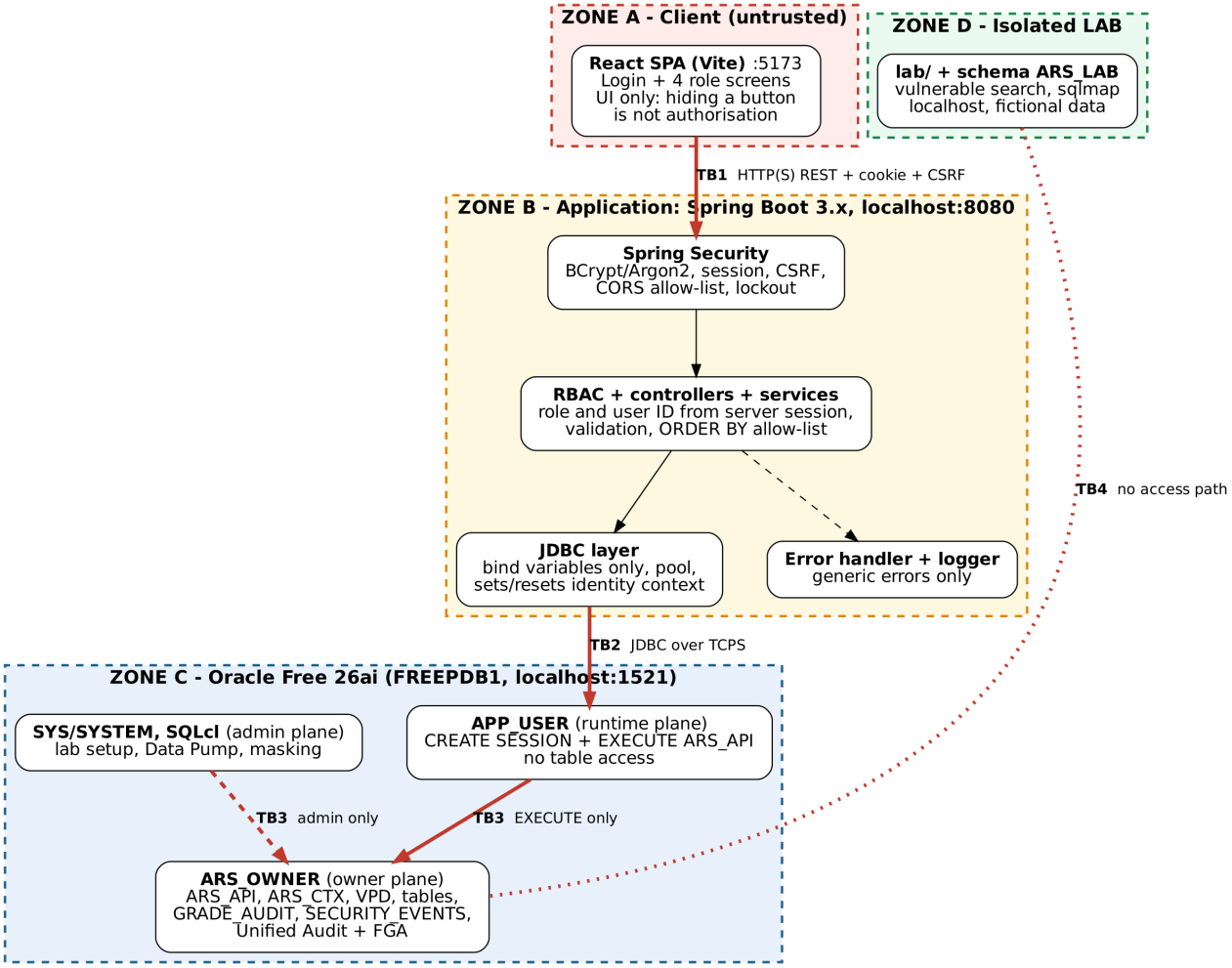


Figure 1 - Architecture overview: React client, Spring Boot application, Oracle database (runtime / owner / admin planes) and the isolated LAB. Red arrows are trust-boundary crossings.

1.1 Inside the Oracle database

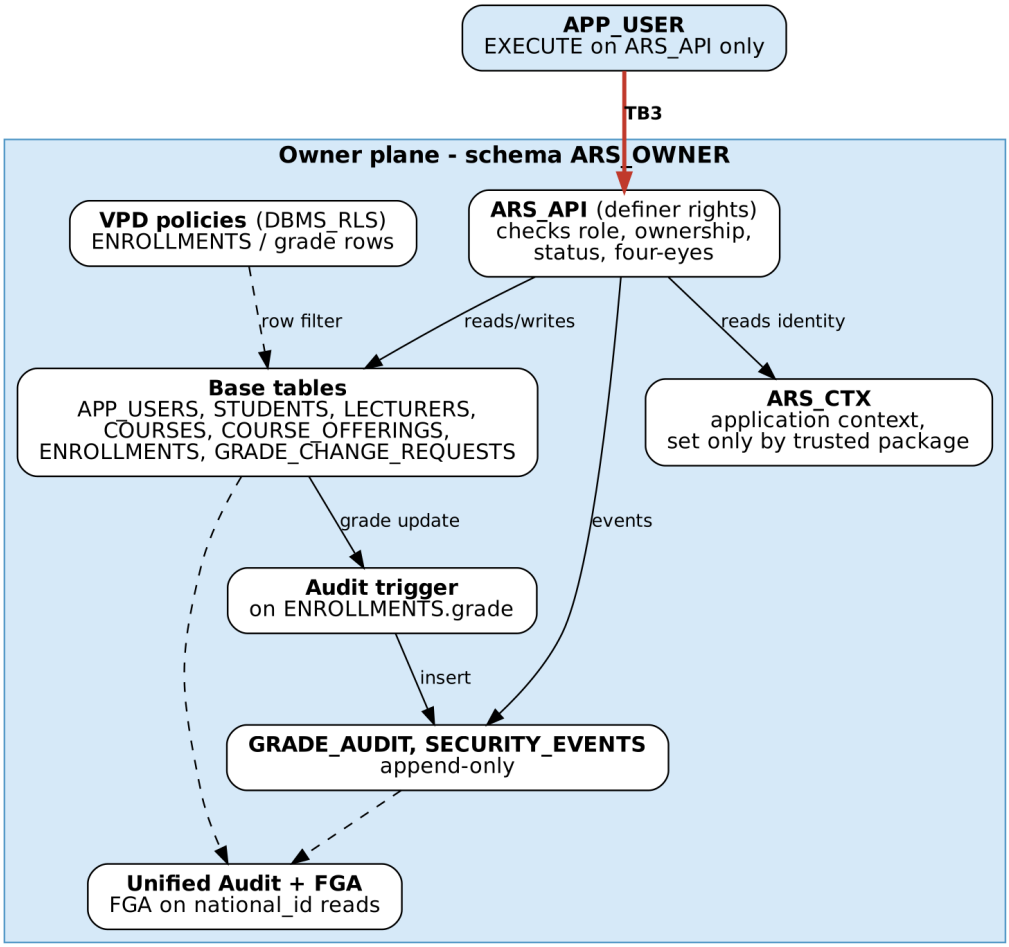


Figure 2 - Owner plane: all access goes through ARS_API; VPD filters rows; the audit trigger and package are the only writers of the audit tables.

2. Trust boundaries

ID	Boundary	What crosses it and how it is controlled
TB1	Client zone <-> Application zone	Browser / React SPA (untrusted, user-controlled) crosses into Spring Boot on localhost:8080. Everything arriving here is untrusted input: credentials, IDs, role claims, search text. Controls: authentication, CSRF, CORS allow-list, RBAC, validation.
TB2	Application zone <-> Database zone	Spring Boot connects to the Oracle listener (localhost:1521, service FREEPDB1) as the limited account APP_USER over a protected channel (TCPS/TLS). The database cannot independently verify the end user behind a pooled connection. Controls: APP_USER least privilege, ARS_API package, context reset, TCPS.
TB3	Runtime plane <-> Owner/Admin plane (inside Oracle)	APP_USER may only EXECUTE ARS_API. Tables, VPD policies, audit tables and triggers belong to ARS_OWNER; SYS/SYSTEM are used only for lab setup and Data Pump. Controls: no direct table grants, definer-rights package, SoD.
TB4	Production-like schema <-> Isolated LAB	The deliberately vulnerable search and the sqlmap scan run only in ARS_LAB / lab/ on localhost with fictional data, never against ARS_OWNER (FR12).

2.1 Components and ports

Component	Technology	Port / service	Trust
React front end	React + Vite	localhost:5173	Untrusted client
Backend API	Spring Boot 3.x, JDK 21	localhost:8080	Trusted code; trusts nothing from the client
Database	Oracle AI Database Free 26ai	localhost:1521, service FREEPDB1 (verify actual name)	Trusted; enforces its own privileges

Component	Technology	Port / service	Trust
LAB	Small vulnerable app + ARS_LAB schema + sqlmap	localhost only, separate port	Deliberately vulnerable; isolated

2.2 Request path of one grade entry (example)

- 1. Lecturer's browser sends POST /api/lecturer/offerings/{id}/grades with session cookie and CSRF token (TB1).
- 2. Spring Security authenticates the session, checks CSRF and CORS origin; @PreAuthorize requires role LECTURER.
- 3. The service validates the grade (0-100) and the offering ID; the lecturer's identity comes from the server session, not from the request.
- 4. The JDBC layer takes a pooled connection as APP_USER (TB2), resets the context, sets the verified identity through ARS_CTX / CLIENT_IDENTIFIER, and calls ARS_API.enter_grade with bind variables.
- 5. ARS_API (TB3 - only EXECUTE is granted) checks that the offering belongs to the caller and is OPEN; VPD restricts rows; the update fires the audit trigger that writes GRADE_AUDIT with the real actor.
- 6. On refusal the database raises an error; the backend maps it to a generic message, writes SECURITY_EVENTS and returns 403 without database details.

2.3 Limits of the trust model (SR04)

APP_USER is one database account shared by every end user through the connection pool. Oracle therefore cannot independently prove which student or lecturer is behind a call: it trusts the identity the backend places in the session context. A bug or compromise in the backend, or a SQL injection that can call ARS_CTX, could assert another user's identity. Mitigations in the design: the context can be set only through the trusted package ARS_CTX; the context is cleared whenever a pooled connection is reused; APP_USER can execute only ARS_API (not arbitrary SQL); every identity assignment and refusal is logged; and the limits are tested and reported rather than hidden.

3. Data flow diagram - level 0 (context)

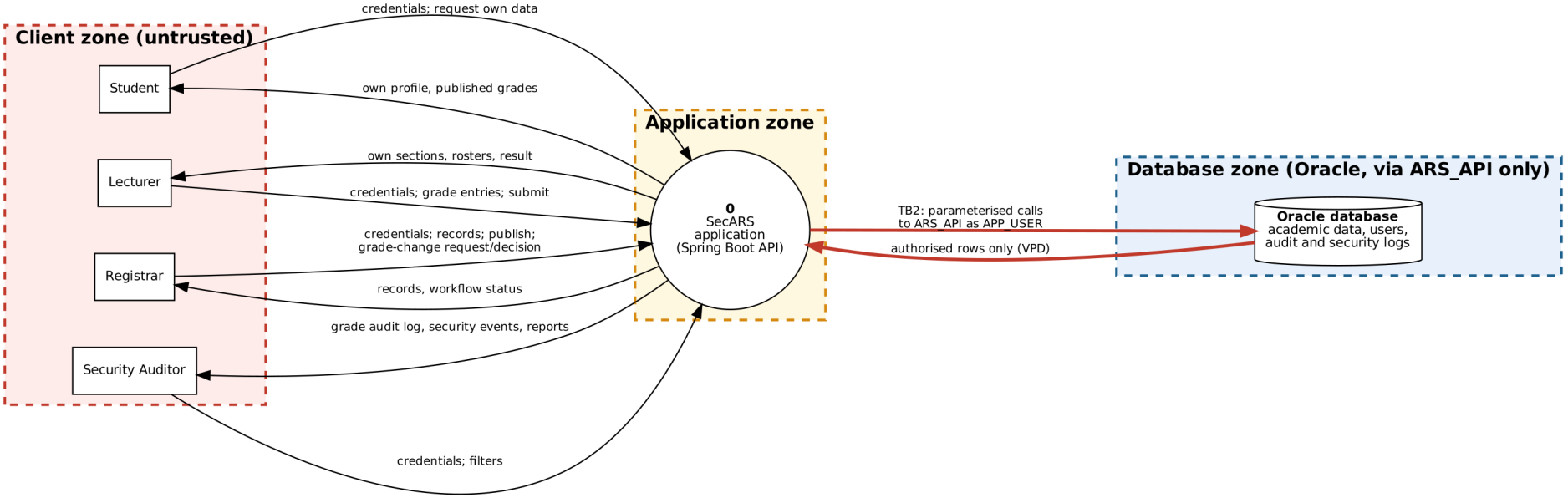


Figure 3 - Context DFD: four role-based external entities, one application, one database. Red = flows crossing TB2/TB3.

4. Data flow diagram - level 1

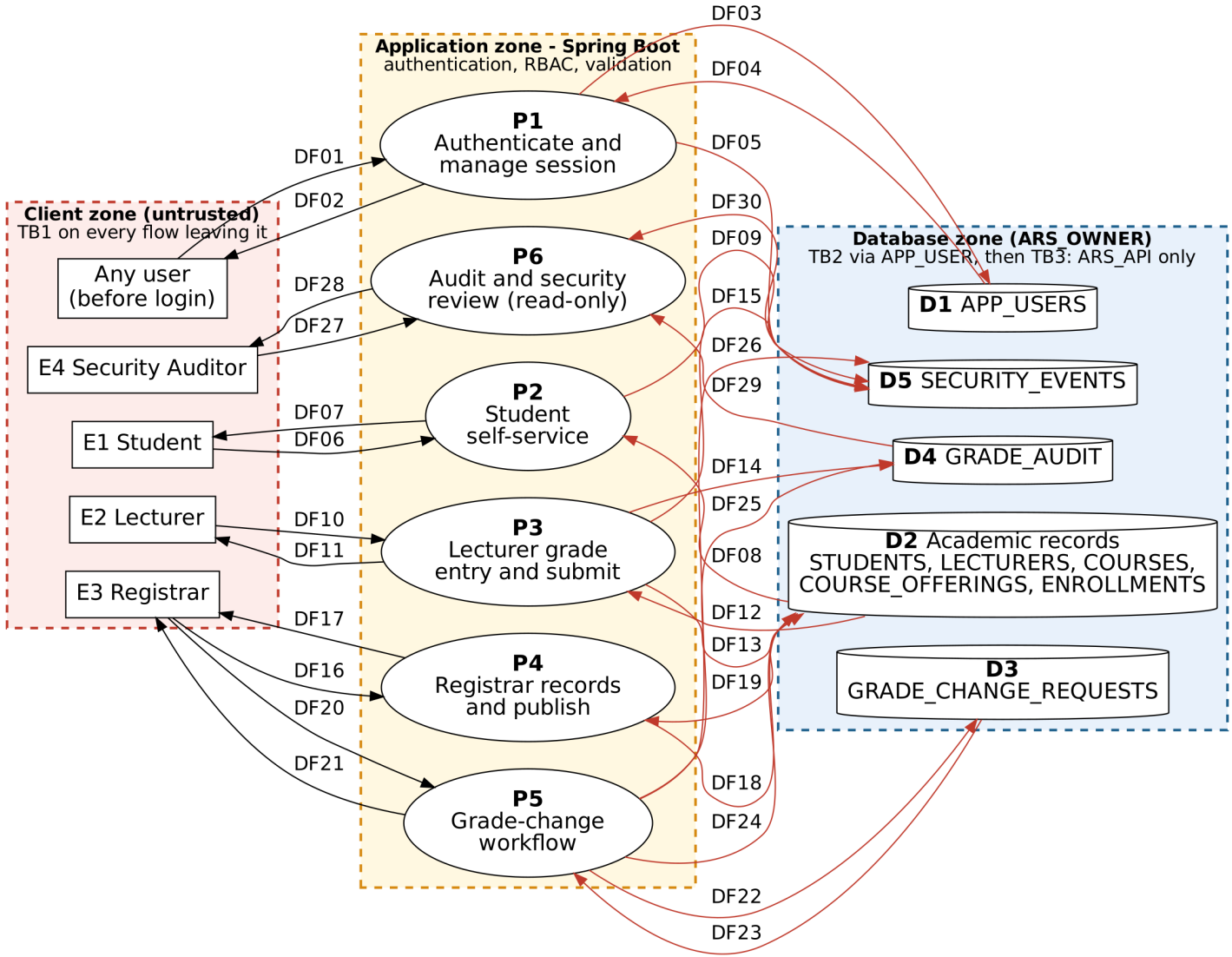


Figure 4 - Level-1 DFD. Processes P1-P6 run in the application zone; data stores D1-D5 live in the owner schema and are reachable only through ARS_API (TB2 then TB3). Black flows cross TB1; red flows cross TB2/TB3.

5. DFD data-flow catalogue

Flow	From	To	Data carried	Boundaries crossed
DF01	E0	P1	Credentials (username, password)	TB1
DF02	P1	E0	Session cookie + role (role comes from the server)	TB1
DF03	P1	D1	Look up user and hash; update failed counter / lock	TB2, TB3
DF04	D1	P1	Hash, role, active flag, lock state	TB2, TB3
DF05	P1	D5	Login success / failed / locked event	TB2, TB3
DF06	E1	P2	Request own data (no client-supplied student ID)	TB1
DF07	P2	E1	Own profile, enrolments, PUBLISHED grades	TB1
DF08	D2	P2	Own rows only (VPD)	TB2, TB3
DF09	P2	D5	IDOR / denied-access event	TB2, TB3
DF10	E2	P3	Grade entries, submit command	TB1
DF11	P3	E2	Own sections, rosters, result	TB1
DF12	D2	P3	Own sections and roster	TB2, TB3
DF13	P3	D2	Grade (own section, OPEN); status to SUBMITTED	TB2, TB3
DF14	P3	D4	ENTRY / EDIT audit record	TB2, TB3
DF15	P3	D5	Denied attempts (other section, closed section)	TB2, TB3
DF16	E3	P4	Student, course, section, enrolment data; publish command	TB1
DF17	P4	E3	Records and results	TB1
DF18	P4	D2	Create / update records; status to PUBLISHED	TB2, TB3
DF19	D2	P4	Records	TB2, TB3
DF20	E3	P5	Grade-change request with reason; approve / reject decision	TB1
DF21	P5	E3	Workflow status	TB1
DF22	P5	D3	Create request; record decision	TB2, TB3
DF23	D3	P5	Pending request	TB2, TB3
DF24	P5	D2	Apply approved grade	TB2, TB3
DF25	P5	D4	POST_CLOSE audit record	TB2, TB3
DF26	P5	D5	Denied event (e.g. self-approval attempt)	TB2, TB3
DF27	E4	P6	Filters (date, actor, type)	TB1
DF28	P6	E4	Grade audit log, security events, aggregate reports	TB1
DF29	D4	P6	Audit records (read)	TB2, TB3
DF30	D5	P6	Security events (read)	TB2, TB3

6. DFD element catalogue

Element	Name	Description	Zone / boundary
E0-E4	Any user, Student, Lecturer, Registrar, Security Auditor	External entities using the React client	Client zone, untrusted (TB1)
P1	Authenticate and manage session	Verify hash, lockout, create/destroy session, return role	Application
P2	Student self-service	Own profile, enrolments, PUBLISHED grades	Application
P3	Lecturer grade entry and submit	Own sections, rosters, grade entry in OPEN, submit	Application
P4	Registrar records and publish	CRUD on students, courses, sections, enrolments; publish	Application
P5	Grade-change workflow	Request with reason; approve/reject by a different registrar	Application
P6	Audit and security review	Read-only access to GRADE_AUDIT, SECURITY_EVENTS, reports	Application
D1	APP_USERS	Accounts, hashes, roles, lockout data	Database, owner plane (TB2, TB3)
D2	Academic records	STUDENTS, LECTURERS, COURSES, COURSE_OFFERINGS, ENROLLMENTS	Database, owner plane (TB2, TB3)
D3	GRADE_CHANGE_REQUESTS	Pending and decided post-closure requests	Database, owner plane (TB2, TB3)
D4	GRADE_AUDIT	Append-only grade-change log	Database, owner plane (TB2, TB3)
D5	SECURITY_EVENTS	Failed logins, lockouts, denied access	Database, owner plane (TB2, TB3)

6.1 LAB data flow (TB4)

The LAB is a separate mini-application bound to localhost with its own account and the schema ARS_LAB containing fictional rows. Flow: tester/sqlmap -> vulnerable search endpoint -> ARS_LAB tables (demonstration of in-band and blind injection), then the same endpoint rebuilt with PreparedStatement -> same payload fails (T7, T8). No flow exists between the LAB and ARS_OWNER; the LAB account has no privilege on ARS_OWNER objects and APP_USER has none on ARS_LAB.